



THE COMPLETE KUBERNETES GUIDE

Architecture · Namespaces · Workloads · Networking · Labs — one coherent volume, deduplicated and rebuilt from MASTER + k8s 2/k8a. The brain, the muscle, the apartments, the traffic — and the hands-on labs that prove it.

 **API Server**
the front door

 **etcd**
cluster memory

 **Scheduler**
placement

 **Controllers**
reconcile

 **kubelet**
node agent

 **Runtime**
runs containers

 **kube-proxy**
service networking

 **Pods**
your apps

PART 0 · ORIENTATION

- ☐ **CH 00** What is Kubernetes? p.4
- ☐ **CH 00-b** What is Kubernetes? (cont.) p.5

PART I · THE CLUSTER

- ☐ **CH 01** What is a Kubernetes Cluster? p.6
- ☐ **CH 02** The Control Plane (Master) p.7
- ☐ **CH 03** The Worker Node p.8
- ☐ **CH 04** How Master & Worker Communicate p.9

PART II · SEE IT WORK

- ☐ **CH 05** Walkthrough — Deploying nginx p.10
- ☐ **CH 06** Self-Healing — When a Worker Dies p.11
- ☐ **CH 07** Can the Control Plane also be a Worker? ... p.12
- ☐ **CH 08** Master : Worker Ratio & etcd Quorum p.13
- ☐ **CH 09** The Diagram Map + Learning Roadmap ... p.14

PART III · NAMESPACES

- ☐ **CH 10** Namespaces — The Big Idea p.15
- ☐ **CH 11** Names Are Scoped + The Three Superpowers p.16
- ☐ **CH 12** Service DNS + Default Namespaces p.17
- ☐ **CH 13** The Complete Namespace Picture p.18

PART IV · WORKLOADS

- ☐ **CH 14** ReplicaSet — What Is It? p.19
- ☐ **CH 15** Labels, Death & Why Deployment p.20
- ☐ **CH 16** The Pod Lifecycle — Owned vs Standalone p.21
- ☐ **CH 17** kubectl — The Command Map p.22
- ☐ **CH 18** Pod Deep Dive p.23
- ☐ **CH 19** Multi-Container Pods p.24
- ☐ **CH 20** Deployment — Rolling Updates p.25
- ☐ **CH 21** Rollout Commands p.26

PART V · YAML & DEBUGGING

- ☐ **CH 22** YAML Anatomy — The Two Specs p.27
- ☐ **CH 23** Debugging 1 — Reading Pod States p.28
- ☐ **CH 24** Debugging 2 — The Toolbox p.29

- API Server ■ etcd ■ Scheduler ■ Controllers ■ kubelet ■ Runtime ■ kube-proxy ■ Namespace
- Secret ■ Pod



How to read this book: one spine — **User** → **kubectl** → **API Server** → **Admission** → **Scheduler** → **Node** → **kubelet** → **Runtime**. Part 0 orients you, I–II build the picture, III–VI add the depth, VII proves it with your hands, VIII makes it stick.



PART VI • SERVICES & NETWORKING

☐ CH 25 NodePort — The Side Entrance p.30	☐ CH 26 Service Networking — The Virtual IP p.31
☐ CH 27 kube-proxy's Real Job p.32	☐ CH 28 Following ONE Packet — 6 Steps p.33

PART VII • LABS

☐ LAB 1 Observe vs Control p.34	☐ LAB 2 Scale & Self-Healing p.35
☐ LAB 3 Namespaces p.36	☐ LAB 4 Rollout & Rollback p.37
☐ LAB 5 Taints & Tolerations p.38	

PART VIII • PROVE IT

☐ QUIZ Self-Check Quiz p.39	☐ KEY Answer Key + Scoring p.40
☐ GLOSS Glossary p.41	☐ CHEAT One-Page Cheat Sheet p.42
☐ FINALE The Map You Can Draw From Memory p.43	



Kubernetes is an open-source **container orchestrator**: it takes the containers you define and — across a fleet of machines — **deploys them, scales them, heals them, connects them, and updates them without downtime**. You describe what you want. It makes reality match. Forever.

**The Name**

Greek κυβερνήτης (kybernetes) = **helmsman** — the one who steers the ship. Hence the wheel logo  (and the word *cybernetics*).

**k8s**

Programmer shorthand: **K** + the 8 letters in between + **s**. Pronounced "K-eights" — you'll read it a hundred times a day.

**Lineage**

Born at Google from **Borg** (its internal cluster manager, ~2003) → open-sourced **2014** → donated to the **CNCF** with v1.0 in **2015**. Written in Go.

The problem it solves — running ONE container is easy; running a fleet is not

WITHOUT AN ORCHESTRATOR

-  A container crashes at 3am → **you** restart it
-  Ten machines free — which one takes **you** decide the new app?
-  40 containers need to find each other → hand-written configs
-  Upgrade from v1 to v2 → downtime, or a hand-rolled script
-  Black Friday? Log in and start containers **manually**

WITH KUBERNETES

-  Crash → **replaced automatically** (reconciliation — CH 06)
-  New app → **Scheduler** picks the best node (CH 02)
-  Containers find each other **DNS + Services** (CH 12, 26)
-  Upgrades roll **zero downtime** + one-command undo (CH 20–21)
-  Scaling = **change one number** (Lab 2)



The core idea — declarative, not imperative: you don't write steps ("start container, then check, then restart..."). You declare a **wish** ("3 replicas of nginx") and Kubernetes runs a permanent **reconciliation loop**: watch actual → compare to desired → act. This one idea is the engine behind self-healing, scaling, and rollouts — and it returns on almost every page of this book.

What's in the box

 **Self-healing** restarts · replaces

 **Scaling** one command

 **Rollouts + rollback** zero downtime

 **Service discovery + LB** built-in DNS

 **Config & Secrets** injected, not baked

 **Storage orchestration** PVC/PV

Industry gravity: every major cloud runs it as a service — **GKE** (Google) · **EKS** (AWS) · **AKS** (Azure) — and it's the CNCF's first graduated project. Learning Kubernetes is learning the *operating system of the cloud*.



What Kubernetes is NOT: not a programming language · not a PaaS (it ships *building blocks*, not opinions) · not a container **builder** (Docker/CI build images — k8s *runs* them) · not a runtime itself (it drives one — containerd, CRI-O, CH 03) · not a monitoring system (Prometheus & friends watch it). It's the **orchestra conductor** 🎻 — it doesn't play a single instrument, but nothing plays without it.



Heritage flex: Kubernetes didn't invent containers or orchestration — Google had been running **billions of containers a week** on Borg for a decade. Kubernetes is what happened when they distilled that war story into an open-source project the rest of the world could use. You're learning fifteen years of production scars, organized into an API.



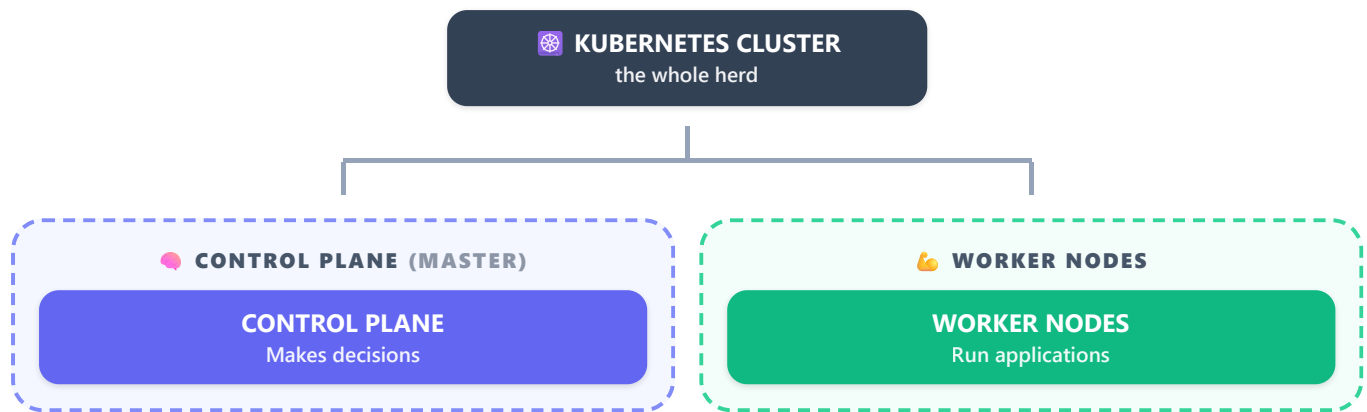
Exam / interview tip: "What is Kubernetes?" has a canonical two-beat answer — **(1) an open-source container orchestrator, (2) declarative:** you declare desired state, it continuously reconciles reality toward it. Add "descended from Google's Borg" for bonus credibility.

🔗 So what does it physically *look like*? Machines split into a brain and muscle — [Chapter 1: What is a Kubernetes Cluster?](#)

↑ back to top



A **cluster** is a collection of machines working together to run your applications. It has exactly two halves:



💡 + 💪 **Think of it as:**



Master = Brain

Thinks, decides,
remembers



Worker = Muscle

Lifts, runs, executes



Cluster = Both

Brain + muscle together



The modern terminology is **Control Plane**, not "Master". You'll still see *master* everywhere — because computer science terminology apparently enjoys **archaeological preservation**. 🏺



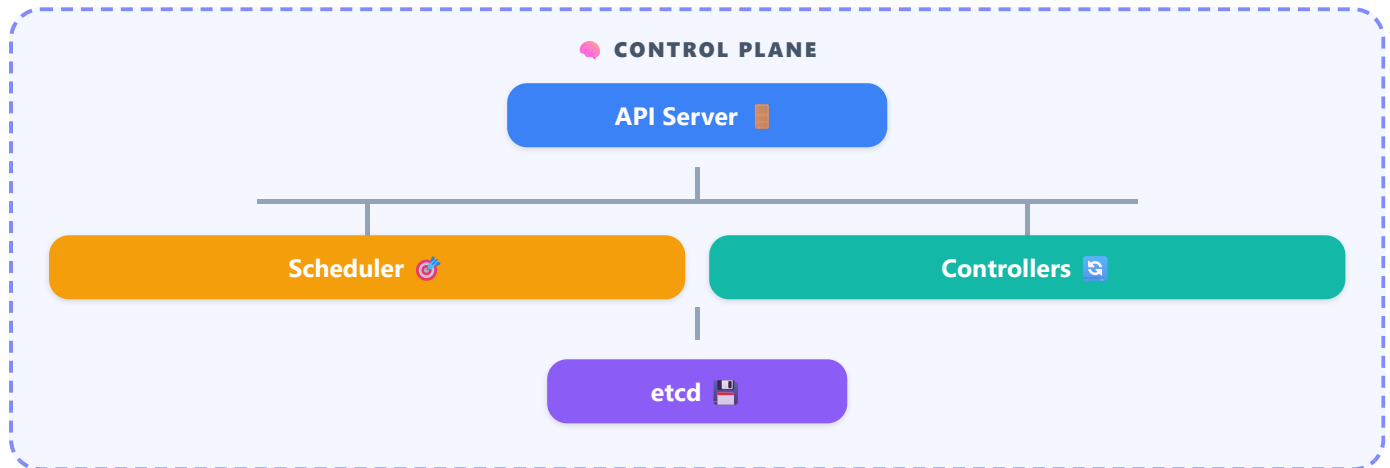
One sentence to remember: Cluster = Control Plane + Worker Nodes. The Control Plane **manages**. The Workers **execute**.

[↑ back to top](#)

The Control Plane **controls the cluster**. It does **not** normally run your application workloads. Its job:

 Receive requests Store cluster information Decide where Pods should run Monitor the cluster Maintain the desired state

Inside the Control Plane



Example — you tell Kubernetes:

```
replicas: 3
```

The Control Plane then figures out:

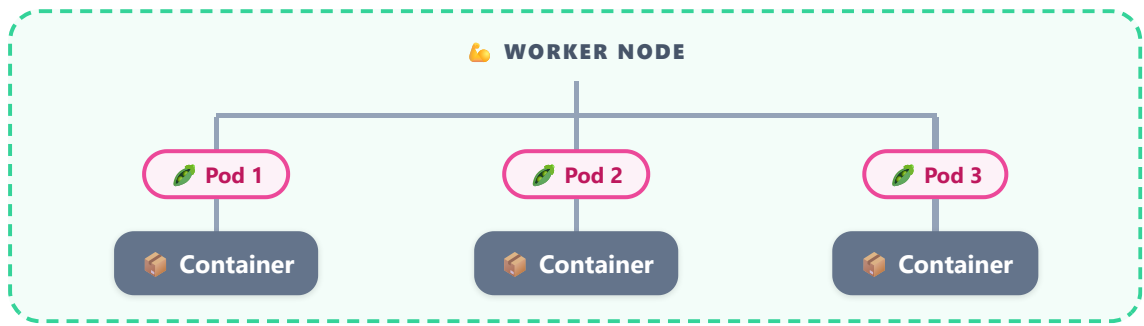
 "I need **3 Pods**." "**Where** should they run?" "Are they **actually** running?" "Did one **die**?" "Do I need to **create another**?"

The four components, one line each: **API Server** = the only door  · **etcd** = the memory  · **Scheduler** = the placement officer  · **Controllers** = the watchdogs . Everything else in the cluster talks *through* the door.

[↑ back to top](#)

A Worker Node is a machine that **actually runs your application workloads**.

 Pods on a Worker



 What's inside a typical Worker Node

WORKER NODE 

|

kubelet



the node agent

|

container runtime



runs containers

|

kube-proxy



service networking

|

Pods



your applications

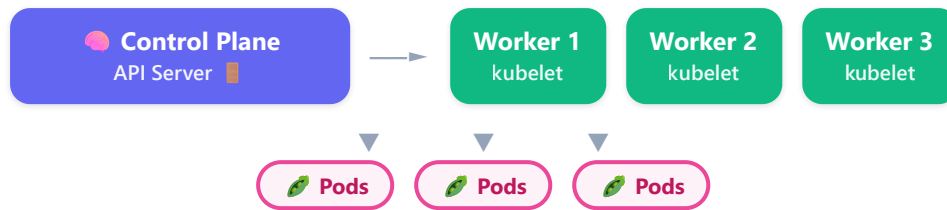
 **The important idea:** the Worker is **where your application actually executes**. The brain thinks — the muscle lifts.

Component	Job	Vibe
kubelet	Manages Pods on this node, reports to the API Server	 Node agent
Container Runtime	Pulls images & runs containers (containerd, CRI-O)	 Engine
kube-proxy	Programs Service networking rules on this node	 Network helper
Pods	Host your containers	 Application unit

[↑ back to top](#)



This is important. The main communication path is the **API Server** — one door, everything walks through it.



The **kubelet** on each worker talks only to the **API Server**. No worker ever talks to etcd, the Scheduler, or the Controllers directly — everything goes through the front door. 🚪

Who	Talks to	Why
kubectl (you)	API Server	Submit/read everything
kubelet	API Server	Get assigned Pods · report status
Scheduler	API Server	Watch unscheduled Pods · write its decisions
Controllers	API Server	Watch actual state · drive it toward desired
API Server	etcd	The <i>only</i> component that touches the memory 💾



Why this design wins: one door = one place for authentication, authorization, and audit. Watch anything, change anything — always through the API.

[↑ back to top](#)



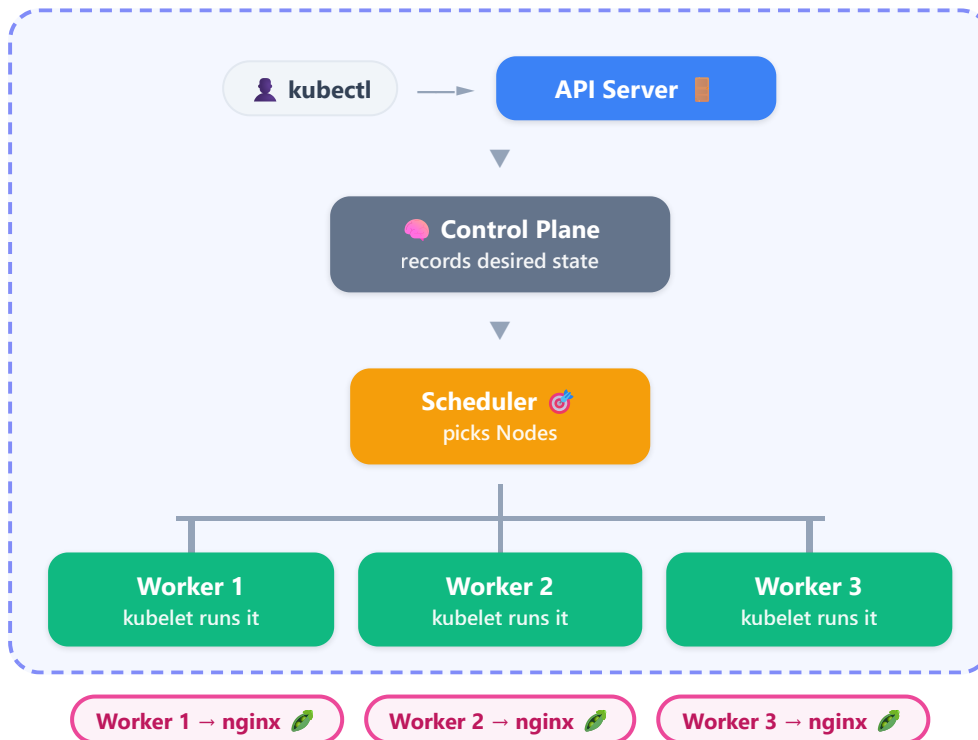
Imagine a cluster with **1 Control Plane** and **3 Workers**:



You run:

```
$ kubectl create deployment nginx --image=nginx --replicas=3
deployment.apps/nginx created
```

The flow



The Control Plane decides. **The Workers execute.**

[↑ back to top](#)



This is where Kubernetes starts **earning its paycheck**. 💰

Worker 1 ❌

Worker 2 ✅

Worker 3 ✅

One of your Pods disappeared. The Control Plane notices a mismatch:

DESIRED

1 2 3

ACTUAL

1 2

Desired 3 ≠ Actual 2 ❌**Controllers react**

Kubernetes creates replacement Pods on healthy workers — assuming workload & cluster resources allow it.

After reconciliation

Worker 1 ❌

Worker 2 ✅
🍷 nginx · 🍷 nginxWorker 3 ✅
🍷 nginx · 🍷 nginx

The reconciliation loop is the heart of Kubernetes: *watch actual → compare to desired → act → repeat, forever.* Nobody "restored" anything — the system simply cannot tolerate a mismatch.



Honest limits: if the cluster is out of CPU/memory, or all healthy nodes are full, replacements stay **Pending**. Self-healing repairs state — it doesn't conjure capacity.

[↑ back to top](#)



Technically yes. In production: no. The mechanism that separates them is a **taint**.

🏠 MINIKUBE — YES, BY DESIGN

A single-node cluster where **one machine is both brain and muscle**. The Control-Plane taint is removed so workloads can land there. Perfect for learning.

```
$ kubectl get nodes
```

NAME	STATUS	ROLES
minikube	Ready	control-plane,master

🏢 PRODUCTION — NEVER

Control-Plane nodes carry a taint like `node-role.kubernetes.io/control-plane:NoSchedule` — the Scheduler **refuses to place normal Pods there**. The brain is protected.

```
$ kubectl describe node cp1 | grep Taints
```

```
Taints: node-role.kubernetes.io/control-plane:NoSchedule
```



Why the isolation matters: a runaway workload on a Control-Plane node can starve the API Server or etcd — and then **the entire cluster's brain** slows or freezes. etcd is especially latency-sensitive.



Remember: "Control Plane ≠ Worker" is not a law of nature — it's a **taint**. Remove the taint and the brain will do deadlifts. (Lab 5 covers taints hands-on.)

[↑ back to top](#)



Why does everyone run **3 Control Planes**? Because etcd (the memory ) needs a **quorum** — a majority of members alive and agreeing — to accept any write.

THE QUORUM MATH

```
quorum = floor((n-1)/2) + 1 members required
# of Control Planes → failures tolerated before the cluster freezes:
1 CP → tolerate 0 failures (one crash = frozen)
3 CP → tolerate 1 failure ← the production minimum
5 CP → tolerate 2 failures
7 CP → tolerate 3 failures
```

Control Planes	Quorum needs	Tolerates	Verdict
1	1 of 1	0	Dev only
2	2 of 2	0	Worst choice — same tolerance as 1, double the cost
3	2 of 3	1	✓ Standard production
5	3 of 5	2	Large/critical clusters



The trap: 2 is the worst number. Two members need *both* alive for quorum — one failure freezes writes, exactly like having one. Even numbers add cost without adding tolerance.



Workers scale independently. The ratio isn't "1 master per N workers" — it's **3–5 Control Planes** (for quorum) plus *as many Workers as your workload needs*. A lost quorum means the cluster keeps running what's deployed but accepts **no new state**.

[↑ back to top](#)



Everything from Part I–II on one canvas. If you can redraw this from memory, you own the architecture.

CLUSTER

- |  **CONTROL PLANE** ×3 for quorum
 - | **API Server**  the only door
 - | **etcd**  state – API Server's alone
 - | **Scheduler**  assigns Pods → Nodes
 - | **Controllers**  reconcile desired ↔ actual
- |  **WORKERS**
 - | **kubelet**  agent – talks to API Server
 - | **Runtime**  pulls + runs containers
 - | **kube-proxy**  Service rules
 - | **Pods** 



The learning order that works



- 1 **Components** — who does what (Parts I–II) · you are here 
- 2 **Namespaces** — carve logical apartments (Part III)
- 3 **Workloads** — ReplicaSet → Deployment → updates (Part IV–V)
- 4 **Networking** — how traffic actually reaches a Pod (Part VI)
- 5 **Labs** — break things, watch them heal (Part VII)



Study rule: never learn a component without knowing *who it talks to*. Isolated component lists evaporate; **communication maps** stick.

[↑ back to top](#)



Without namespaces, one cluster = one giant bucket. Frontend, AI and dev all collide — same names, no boundaries. A namespace is the cluster saying **"I can carve logical apartments inside one building."**

PROBLEM — ONE FLAT BUCKET

Cluster

```
| pod-a
| pod-b  frontend?
| pod-c  ai?
| svc-a   # who owns what?
```

SOLUTION — LOGICAL APARTMENTS

Cluster

```
| frontend Pods + Services
| ai       Pods + Services
| dev      Pods + Services
same nodes, same control plane
```

When namespaces shine**Teams**

frontend / backend / ai

**Environments**dev / staging / prod via
namespaces**Apps**

app-a / app-b isolated, same infra



Namespace = logical isolation — it affects names, RBAC, quotas. It does **not** carve hardware or network by itself.



Environment ≠ Namespace. Environment is the concept; namespace is one object you can implement it with (another: separate clusters).

[↑ back to top](#)

Names Are Scoped + The Three Superpowers

A Pod's full identity is **namespace + kind + name**. So `dev/nginx` and `prod/nginx` are two different objects — no collision.

```
Cluster
├─ dev
│  └─ nginx Pod ← identity: dev / Pod / nginx
└─ prod
    └─ nginx Pod ← identity: prod / Pod / nginx
```

⚡ The three superpowers namespaces unlock

🏠 RBAC — KEYS

"This team may act **only inside frontend**." Permissions scoped per namespace.

📊 QUOTAS — BUDGET

"ai may use at most **4 CPU / 8Gi**, 10 Pods." A ResourceQuota caps each apartment's totals.

🚧 NETWORKPOLICY — DOOR RULES

"Pods in dev may **not** call prod." Traffic rules scoped per namespace.



Not everything lives in an apartment. Nodes, ClusterRoles, PersistentVolumes and Namespaces themselves are *cluster-scoped* — `kubectl get nodes -n dev` silently ignores the `-n`. Nodes belong to the building, not an apartment.



The rule: namespaced → identity includes the namespace · cluster-scoped → namespace flags do nothing. **Namespaces partition names, not hardware.**

↑ back to top



How Pods find each other: DNS

Every Service gets a DNS name built from its namespace — that's how cross-Pod traffic finds its way:

nginx-service
Service



nginx-service .frontend .svc.cluster.local

same short name resolves *within* the same namespace — "nginx-service" from inside frontend just works

frontend
nginx-service

ai
nginx-service

← same Service name, two namespaces, zero collision

The namespaces you always have

```
$ kubectl get namespaces
NAME                STATUS
default             Active # you land here when you omit -n
kube-system         Active # the cluster's own components
kube-public         Active # public info
kube-node-lease     Active # node heartbeats
```

📌 `kubectl get pods` is secretly `kubectl get pods -n default`

📌 `kube-system` is where the Control Plane's own Pods live (on kubeadm clusters)



Mailbox rule: "nginx-service, Apt frontend" vs "nginx-service, Apt ai" — same name, different mailbox, and the post office (DNS + API Server) always delivers correctly.

↑ back to top



CLUSTER

```

├─ Namespace: frontend  Pods · Services · Quotas
├─ Namespace: ai        Pods · Services · Quotas
├─ Namespace: dev       Pods · Services · Quotas
├─
└─ Nodes # OUTSIDE every namespace
    ├─ node1 (4 CPU / 8Gi)
    └─ node2 (4 CPU / 8Gi) # one shared pool

```

Layer	Shared across namespaces?	Notes
Control Plane (API Server, etcd, Scheduler)	✓ Yes — one per cluster	A namespace is just a field on objects
Nodes · CPU · Memory · Network	✓ Yes	Quotas <i>limit</i> usage, they don't carve hardware
Pods · Services · ConfigMaps	✗ No — namespaced	dev/nginx ≠ prod/nginx



Namespace ≠ separate cluster. Same control plane, same Nodes, same CPU/memory pool. If an exam asks "is a namespace its own cluster?" — the answer is a hard **NO**.



Apartment summary: cluster = building · namespace = apartment · quota = "max electricity for this apartment" · RBAC = who holds which keys · Ingress = the front desk routing visitors. One metaphor, whole book.

[↑ back to top](#)



A ReplicaSet is **the guarantor**: it watches a set of Pods that carry specific **labels**, and keeps their count at the number you asked for. Not one more, not one fewer.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3           # ← the promise
  selector:
    matchLabels:
      app: nginx        # ← who it manages
  template:             # ← the pod stamp it uses
    metadata:
      labels: { app: nginx } # must match the selector!
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

How it "counts"

 **Selector** — the ReplicaSet claims every Pod whose labels match `app: nginx`

 **Compare** — desired 3 vs actual (say) 2

 **Act** — create one more Pod from the template (or delete surplus)

 **Repeat** — this is just the reconciliation loop again, one level down



Label match is everything. If the template's labels don't satisfy the selector, the ReplicaSet refuses to start. And any *stray* Pod with matching labels gets adopted — willingly or not.



ReplicaSet = labels + count + template. You'll rarely write one by hand — a **Deployment** creates them for you (CH 15).

[↑ back to top](#)



STANDALONE POD DIES



Gone. Forever. Nobody holds a copy of the wish — the Pod's spec died with it. This is why "just run a Pod" is a demo trick, not a deployment.

OWNED POD DIES



Recreated automatically. The ReplicaSet still holds the template — the mismatch triggers a replacement with a **new name** (e.g. `nginx-rs-x7k2p`).

So why isn't ReplicaSet enough? The ownership chain

Deployment you talk to this one

└─ **ReplicaSet v1** manages Pods at image `nginx:1.25`

└─ pod

└─ pod

└─ pod

change the image → Deployment creates **ReplicaSet v2**, rolls Pods over ← CH 20

ReplicaSet answers: " are the right number of Pods running? "

Deployment answers: " are they running the right version? — and can we switch versions with zero downtime? "



Own vs run: a standalone Pod is *run*; an owned Pod is *guaranteed*. The Deployment exists because the ReplicaSet can keep count — but it can't change what it's counting *without downtime*. The Deployment can.

↑ back to top

The five-second experiment that makes ownership real. Two terminals:

TAB 1 · WATCH

```
$ kubectl get pods -w
```

NAME	READY	STATUS
web-7d9f	0/1	ContainerCreating
web-7d9f	1/1	Running
web-7d9f	0/1	Terminating
web-x7k2p	0/1	ContainerCreating
web-x7k2p	1/1	Running # new name!

TAB 2 · ACT

```
$ kubectl delete pod web-7d9f
pod "web-7d9f" deleted
# watch TAB 1: a replacement appears
# same deployment, brand-new pod name
```

What we just proved

- ✓ The Pod's **name is disposable** — the ReplicaSet's **promise is not** .
- ✓ Deletion is just another mismatch: desired 1, actual 0 → create.
- ✗ A **standalone** Pod (no owner) deleted = silence. No replacement. No drama.

💡 **Status decoding:** **ContainerCreating** = image pulling + CNI wiring · **Running** = process up · **Terminating** = shutting down gracefully · **Pending** = not yet placed (CH 23).

🔑 **Owned vs standalone is the single most important workload concept.** Every "why did my Pod come back?!" mystery is answered by: *someone owns it* — find the ReplicaSet.

[↑ back to top](#)



Job	Command	What it does
Create	<code>kubectl create deployment nginx --image=nginx</code>	Imperative quick-start
	<code>kubectl apply -f file.yaml</code>	Declarative — the production way
Inspect	<code>kubectl get pods/dep/rs/svc -n ns</code>	List · add <code>-w</code> to watch live · <code>-o wide</code> for Node/IP
	<code>kubectl describe pod NAME</code>	Full story — events at the bottom
	<code>kubectl logs NAME [-c container]</code>	App output · <code>-f</code> follow · <code>--previous</code> for crashed
Debug	<code>kubectl exec -it NAME -- sh</code>	Shell inside the container
	<code>kubectl get events --sort-by=.lastTimestamp</code>	Cluster-wide recent history
Modify	<code>kubectl scale deploy nginx --replicas=5</code>	Change the count
	<code>kubectl set image deploy/nginx nginx=nginx:1.26</code>	Trigger a rolling update
	<code>kubectl rollout status/undo/history deploy/nginx</code>	Watch / revert / list revisions
Delete	<code>kubectl delete pod/dep/ns NAME</code>	Deleting an ns deletes everything inside it ⚠️

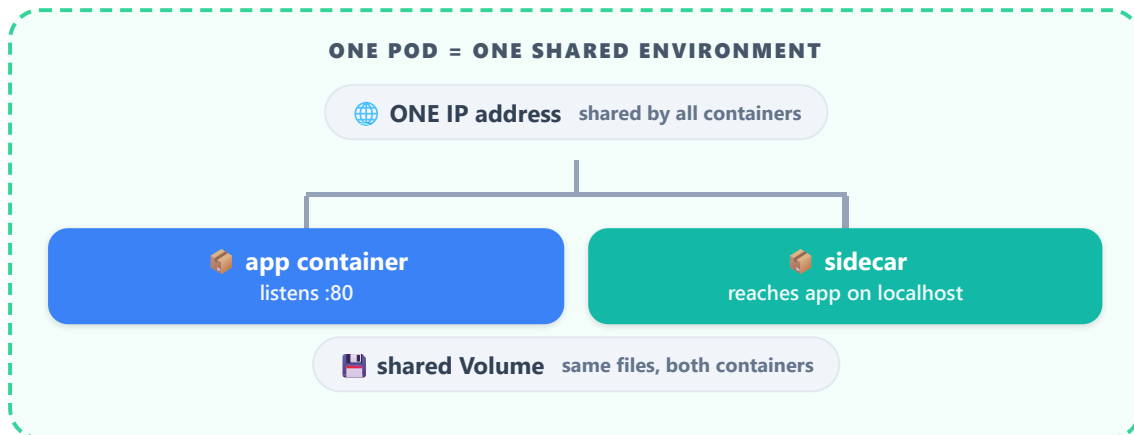


The mental model: kubect! is just a **client of the API Server**. Every command above is an HTTP call to the front door — nothing magical, nothing direct.

[↑ back to top](#)



The Pod is the **smallest deployable unit** — not the container. A Pod is a **co-located family of containers** that share fate.



🌐 **One IP per Pod** — containers inside talk via `localhost:port`, never "find the other container's IP"

💾 **Shared volumes** — declared in the Pod spec, mounted by any of its containers

⌚ **Shared lifecycle** — born together, scheduled together (same Node), die together

🚫 **Never scale "a container"** — you scale Pods; want more copies → more Pods



Port clashes are real: two containers in one Pod cannot bind the same port — they literally share the network stack.



The test for Pod-membership: *must these containers live and die together on the same machine?* Yes → one Pod. No → separate Pods (and let the Scheduler place them freely).

↑ back to top



The classic pattern: the **sidecar** — a helper container riding along with the app container inside one Pod.

```
spec:
  containers:
  - name: app
    image: myapp:2.0
    volumeMounts:
    - { name: logs, mountPath: /var/log/app }
  - name: log-shipper      # the sidecar
    image: busybox
    command: ["sh", "-c", "tail -F /logs/app.log"]
    volumeMounts:
    - { name: logs, mountPath: /logs } # same volume!
  volumes:
  - name: logs
    emptyDir: {}
```



Good vs bad Pod-mates

✓ BELONG TOGETHER

- 📄 app + log shipper (shared files)
- 🔒 app + auth proxy (shared localhost)
- 🔄 app + config reloader (shared lifecycle)

✗ DON'T

- frontend + database (scale & fail independently)
- two unrelated services (one crash restarts both... and they fight over ports)



Heuristic: if the two containers *could run on different machines*, they belong in **different Pods**. Pod-mates are glued on purpose — glue is a cost.

[↑ back to top](#)



Change a Deployment's image and it doesn't rebuild Pods in place — it **creates a new ReplicaSet** and rolls traffic from old to new, keeping availability the whole time.

```
$ kubectl set image deploy/nginx nginx=nginx:1.26
deployment.apps/nginx image updated
$ kubectl rollout status deploy/nginx
Waiting for deployment "nginx" rollout to finish...
deployment "nginx" successfully rolled out
```



What actually happened

Deployment nginx

```
|— ReplicaSet v2 nginx:1.26   scales 0 → 3 🙌
|   |— 🍃 🍃 🍃   new Pods replace old
|— ReplicaSet v1 nginx:1.25   scales 3 → 0 🙋 (kept for rollback)
```

⚖️ The Controller walks the count: add a v2 Pod → remove a v1 Pod → repeat

📄 Service selectors keep pointing at labels — traffic slides over automatically

📄 v1's ReplicaSet is *retained* (scaled to 0) — that's your rollback insurance (CH 21)



Zero downtime is the point: at every instant, enough healthy Pods serve traffic. The Deployment turned "risky big-bang upgrade" into "boring arithmetic."

↑ back to top

```
$ kubectl rollout history deploy/nginx
REVISION  CHANGE-CAUSE
1         <none>
2         nginx:1.26
$ kubectl rollout undo deploy/nginx
deployment.apps/nginx rolled back
$ kubectl rollout status deploy/nginx
successfully rolled out # ← back on :1.25
```

-  `rollout status` — watch a rollout finish (script-friendly)
-  `rollout history` — every revision is just an old ReplicaSet with a number
-  `rollout undo` — flip back to the previous template · `--to-revision=1` to pick
-  `revisionHistoryLimit: 3` — how many old ReplicaSets to keep (default 10)



Undo doesn't rewind time — it creates a **new revision** (v3) with v1's template. History only moves forward; the ReplicaSet list is the audit trail.



The rollout trilogy — `status` / `history` / `undo` — is the exam's favorite Deployment command set. Learn it as one unit.

↑ back to top



Every Kubernetes manifest is the same envelope: **apiVersion** · **kind** · **metadata** · **spec**. The "wish" lives in the spec.

Field	Meaning	Example
<code>apiVersion</code>	Which API group/version serves this kind	<code>v1</code> (Pod) · <code>apps/v1</code> (Deployment)
<code>kind</code>	What object you want	<code>Pod</code> , <code>Service</code> , <code>Deployment</code>
<code>metadata</code>	Name · namespace · labels	<code>name: nginx</code>
<code>spec</code>	The desired state	replicas, containers, ports...



The Two-Specs Mystery (Deployment)

```
spec:                                # the DEPLOYMENT's wish
  replicas: 3
  selector: ...
  template:                          # a whole Pod hiding inside!
    metadata: labels: ...
    spec:                             # ← the POD's wish (spec inside spec)
      containers: ...
```



Outer spec = what the Deployment manages: count, selector, template



Inner spec (`spec.template.spec`) = the actual Pod: images, ports, env, resources



`spec.template` is the **cookie cutter** — the ReplicaSet stamps Pods from it; change anything inside and you trigger a rollout (CH 20)



Indentation is structure in YAML. The most common manifest bug is writing the Pod spec at the wrong nesting level — and the API Server answers with a confusing validation error. Count your two spaces.

[↑ back to top](#)



The STATUS column is a diagnosis. Each state tells you *which stage* failed — before you run a single command.

Status	Meaning	Classic cause	First move
Pending	Accepted, not placed	Not enough CPU/mem · taints · PVC pending	<code>describe pod</code> → events
ContainerCreating	Placed, pulling/wiring	Normal for seconds; stuck = image/CNI issue	<code>describe pod</code> → events
ImagePullBackOff	Can't fetch image	Typo in tag · private registry, no secret	Check image name & pull secret
CrashLoopBackOff	Starts, dies, repeats	App crashes on boot — bad config, missing env	<code>logs --previous</code>
Running	Process is up	— (not necessarily <i>healthy</i> — that's probes)	Test the endpoint
Completed	Finished & exited 0	Job finished — normal for Jobs, odd for servers	Expected? done ✓



The backoff family: `ImagePullBackOff` and `CrashLoopBackOff` are not errors *of* Kubernetes — they're Kubernetes *retrying* after your app/image failed. The real error is one layer deeper, in the events or logs.



Memory hook: Pending = the **Scheduler** said no · ImagePull = the **Registry** said no · CrashLoop = the **App** said no. Three different culprits, three different rooms to search.

↑ back to top



```
$ kubectl describe pod web-x7k2p      # 1 · full biography...
Events:                               # ...always read the BOTTOM
  Failed to pull image "nginx:1.99"
$ kubectl logs web-x7k2p --previous    # 2 · crashed container's last words
panic: missing REQUIRED_DB_PASSWORD
$ kubectl exec -it web-x7k2p -- sh     # 3 · go inside
$ kubectl get events --sort-by=.lastTimestamp # 4 · cluster timeline
```

DESCRIBE — THE BIOGRAPHY

Config as the cluster sees it, plus the **Events block at the bottom** — the single most information-dense place in debugging.

LOGS --PREVIOUS — THE AUTOPSY

CrashLoopBackOff kills the evidence — `--previous` reads the **last dead container's** output. Forgetting this flag = reading a fresh empty log and panicking.

The 4-step playbook (in order)

- 1 **get pods** — read the STATUS column (CH 23 table)
- 2 **describe pod** — events at the bottom name the failing stage
- 3 **logs --previous** — the app's own error message
- 4 **exec -it -- sh** — poke around inside: DNS, env, files, curl



90% of debugging is reading. The cluster narrates its own failures — through events and logs — if you ask in the right order.

[↑ back to top](#)

ClusterIPs are invisible from outside the cluster. **NodePort** punches a hole: every Node opens one high port and forwards it to your Service.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort          # default is ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80            # inside the cluster
      targetPort: 80      # container's port
      nodePort: 30080     # outside – or omit & let k8s pick
```

THE PATH IN



"ANY node" is the superpower — even a Node with **no nginx Pod** opens the door and forwards across.

🔧 Valid range: **30000–32767** (omit `nodePort` and one is auto-assigned)

🔒 Every Node gets the same port — hit `192.168.1.11:30080` or `.13:30080`, same result



NodePort is for demos & edge cases — raw ports, no load balancing across nodes, no TLS story. Production rides **Ingress** (Book 03 😊).

↑ back to top



Here's the trick nobody tells you: the ClusterIP your Service owns is a **lie**. Nothing is listening on it. No process holds it. It's a **virtual IP (VIP)** — and that's the elegance.

RUSSIAN DOLLS — WHERE THE VIP REALLY GOES

10.96.0.50
the VIP — a fiction

→ DNAT →

10.244.1.5
a real Pod IP

packets "to the VIP" are rewritten **in transit** to a real Pod — the VIP never terminates anything

Address ranges — separate drawers:

Service CIDR **10.96.0.0/12** # VIPs live here (fake, routable only via rules)

Pod CIDR **10.244.0.0/16** # real Pod IPs (per-node slices)

🚫 You can't `ping` a ClusterIP — nothing answers; the rules only rewrite packets *passing through*

👤 Pods died and got new IPs? Nobody notices — everyone still dials the VIP

📅 The ranges **must not overlap** — two drawers, zero confusion



The Service is a stable illusion: an eternal name + eternal IP in front of a set of mortals (Pods) that churn constantly. The illusion is maintained by **kube-proxy** — next chapter.

↑ back to top

kube-proxy does **not** forward a single packet. Its entire job is two verbs: **watch & program**.

✗ WHAT IT DOES NOT DO

- Sit in the traffic path
- Proxy connections through itself
- Touch Pod-to-Pod traffic

✓ WHAT IT DOES

- Watch** the API Server: Services + Endpoints
- Program** iptables/IPVS rules on *its* node
- Re-program the instant Pods change

🔮 The rules it writes (iptables + DNAT)

```
# conceptually, on every node:  
if dst == 10.96.0.50:80 # the VIP  
  then rewrite dst → 10.244.1.5:80 # pick an endpoint (random)  
  else if ... next endpoint # (rule set balances across all)
```



The Illusion, resolved: a Pod dies → Endpoints object updates → every node's kube-proxy rewrites its rules → the **VIP never moved**. Stability on the outside, churn on the inside, and the packets never knew.



One-liner for exams: *kube-proxy watches Services/Endpoints and programs packet-rewriting rules on every node — it never forwards traffic itself.*

[↑ back to top](#)



A Pod runs `curl nginx-svc`. Follow the single packet:



- 1 DNS** — `nginx-svc` resolves to the VIP `10.96.0.50` · CoreDNS answered from the Service's namespace
- 2 Send** — Pod sends to the VIP (nothing is there — remember CH 26) · dest: `10.96.0.50:80`
- 3 DNAT** — the node's iptables rules rewrite the destination → a real Pod IP: `10.244.1.5:80`
- 4 Route** — same node → direct delivery; different node → CNI overlay carries it
- 5 Receive** — the Pod sees a normal packet addressed to *its own* IP · it replies
- 6 Return** — conntrack rewrites the reply back through the VIP — the client believes it talked to `10.96.0.50` the whole time 📡



The entire machine in one sentence: DNS gives you a stable fake IP; kube-proxy's rules secretly rewrite it to a mortal Pod; conntrack makes the lie symmetric. **That's Service networking — everything else is detail.**

[↑ back to top](#)



TAB 1 · OBSERVE

```
$ kubectl get deployments -w
$ kubectl get pods -w
$ kubectl get rs -w
```

TAB 2 · CONTROL

```
$ kubectl create deploy web --image=nginx --replica
$ kubectl delete pod <pod-name>
$ kubectl scale deploy web --replicas=4
```

STEP 1 · Create

RUN

```
$ kubectl create deployment web --image=nginx --replicas=2
deployment.apps/web created
```

EXPECTED in TAB 1

```
web 2/2 READY
web-aaaaa 1/1 Running
web-bbbbb 1/1 Running
```

STEP 2 · Kill a Pod

RUN

```
$ kubectl delete pod web-aaaaa
pod "web-aaaaa" deleted
```

EXPECTED

```
web-aaaaa Terminating
web-cccc  ContainerCreating
web-cccc  Running # replacement!
```

STEP 3 · What we proved

A ReplicaSet owns the Pods → deletion is a mismatch, not a tragedy. The **name** died; the **promise** didn't. (Standalone pods: `kubectl run p1 --image=nginx` then delete it — silence. Try it.)

↑ back to top



STEP 1 · Scale to 5

⚙️ RUN

```
$ kubectl scale deploy web --replicas=5
deployment.apps/web scaled
```

👁️ EXPECTED

```
three new pods
ContainerCreating → Running
total: 5/5
```

STEP 2 · Delete two at once

⚙️ RUN

```
$ kubectl delete pod web-cccc web-dddd
```

👁️ EXPECTED

```
# two replacements appear
# concurrently, new random names
back to 5/5 in seconds
```

STEP 3 · The edge — starve it

⚙️ RUN

```
$ kubectl scale deploy web --replicas=50
$ kubectl get pods
```

👁️ YOU'LL SEE

```
many pods stuck
Pending # not enough cluster CPU
```

The lesson: self-healing repairs *state*, not *capacity*. Scale back down: `kubectl scale deploy web --replicas=2`.



Scale is just editing the promise. The ReplicaSet reconciles upward and downward — surplus Pods get terminated, deficits get filled. Same loop, both directions.

[↑ back to top](#)



STEP 1 · Same name, two apartments

⚙️ RUN

```
$ kubectl create ns dev
$ kubectl run nginx --image=nginx -n dev
$ kubectl run nginx --image=nginx
```

👁️ EXPECTED

```
$ kubectl get pods -A | grep nginx
default    nginx    Running
dev        nginx    Running
# two different pods, zero collision
```

STEP 2 · Namespace flags matter

⚙️ RUN

```
$ kubectl get pods          # default only
$ kubectl get pods -n dev   # dev only
$ kubectl get pods -A      # everything
```

👁️ NOTE

```
$ kubectl get nodes -n dev
# same node list – -n is IGNORED
# nodes are cluster-scoped!
```

STEP 3 · The cascade ⚠️

⚙️ RUN

```
$ kubectl delete ns dev
namespace "dev" deleted
```

👁️ EXPECTED

```
# everything inside dev is
# terminated and removed –
# the pod included
```



Deleting a namespace deletes EVERYTHING in it. There is no trash can, no undo. In production, some teams even pin RBAC so only admins can delete namespaces.

[↑ back to top](#)



STEP 1 · Upgrade

⚙️ RUN

```
$ kubectl set image deploy web nginx=nginx:1.26
$ kubectl rollout status deploy web
```

👁️ EXPECTED

```
Waiting for deployment...
successfully rolled out
$ kubectl get rs
# two RS: old 0, new 2
```

STEP 2 · Ship a broken tag

⚙️ RUN

```
$ kubectl set image deploy web nginx=nginx:1.99
$ kubectl rollout status deploy web
```

👁️ EXPECTED

```
Waiting... 1 unavailable
$ kubectl get pods
one pod in ImagePullBackOff
OLD pods still running!
```

STEP 3 · Roll back

⚙️ RUN

```
$ kubectl rollout undo deploy web
$ kubectl rollout history deploy web
```

👁️ EXPECTED

```
successfully rolled out
# bad pod terminated, good ones back
# history shows a NEW revision (v3)
```



The safety net is structural: a bad rollout can never take down the old version — the Deployment only removes old Pods once new ones are Ready. Broken tag? The cluster waits, old version serves, you type `undo` .

[↑ back to top](#)



STEP 1 · Taint AFTER the Pod exists

⚙️ RUN

```
$ kubectl taint node <node> env=prod:NoSchedule
$ kubectl scale deploy web --replicas=4
```

👁️ EXPECTED

```
# existing pods keep running
# NEW 4th pod → Pending
0/1 Nodes available
...taint...that pod tolerates not
```

Key insight: **NoSchedule** only gates *new* placements — it never evicts.

STEP 2 · Rescue with a toleration

```
spec:
  tolerations:
  - key: env
    operator: Equal
    value: prod
    effect: NoSchedule # pod may now LAND on the tainted node
```

Apply → the Pending pod schedules immediately. **Taint = the lock. Toleration = the key.**

STEP 3 · NoExecute — the eviction

⚙️ RUN

```
$ kubectl taint node <node> env=prod:NoExecute
```

👁️ EXPECTED

```
# non-tolerating pods on that
# node get EVICTED – now
```

Effect	Existing Pods	New Pods
NoSchedule	stay	blocked (unless tolerated)
PreferNoSchedule	stay	avoided if possible
NoExecute	evicted	blocked (unless tolerated)



Remember the direction: taints repel (node → pods); tolerations permit (pod → tainted node). Taints **never attract** — for that you'd use nodeSelector/affinity (Book 04 in depth).

↑ back to top



Q1. A Kubernetes cluster consists of...

- A.** API Server + etcd only **B.** Control Plane + Worker Nodes **C.** Nodes + Pods only **D.** Masters only

Q2. Which component is the ONLY door to the cluster — everyone talks through it?

- A.** etcd **B.** Scheduler **C.** API Server **D.** kubelet

Q3. A worker dies running 1 of 3 Deployment replicas. What happens?

- A.** Nothing until you fix the node **B.** Replica count drops to 2 **C.** A replacement Pod is created on a healthy node **D.** The whole Deployment restarts

Q4. With 5 Control-Plane nodes, how many can fail before etcd loses quorum?

- A.** 1 **B.** 3 **C.** 2 **D.** 4

Q5. `kubect1 get nodes -n dev` returns...

- A.** dev's nodes **B.** an error **C.** nothing **D.** all nodes — nodes are cluster-scoped, the flag is ignored

Q6. You delete a ReplicaSet-owned Pod. Result?

- A.** Gone forever **B.** Recreated with a new name **C.** Same Pod restarts **D.** ReplicaSet is deleted too

Q7. Pod stuck in `ImagePullBackOff`. Most likely cause?

- A.** Bad/typo image tag or missing registry credentials **B.** App crashed **C.** No CPU available **D.** Service misconfigured

Q8. Valid NodePort range?

- A.** 1024–65535 **B.** 1–1024 **C.** 8080–9999 **D.** 30000–32767

Q9. kube-proxy's actual job?

- A.** Forward packets between nodes **B.** Assign Pod IPs **C.** Watch Services/Endpoints and program node routing rules **D.** Run CoreDNS

Q10. Which taint effect **evicts existing** non-tolerating Pods?

- A.** NoExecute **B.** NoSchedule **C.** PreferNoSchedule **D.** EvictOnly

↑ back to top



Q	Ans	Why — one line
1	B	Cluster = Control Plane (decides) + Workers (execute) — CH 01
2	C	API Server is the only door; only it touches etcd — CH 02/04
3	C	Desired 3 ≠ Actual 2 → reconciliation creates a replacement — CH 06/16
4	C	Quorum of 5 = 3 → tolerate $\text{floor}(5/2) = 2$ failures — CH 08
5	D	Nodes are cluster-scoped; <code>-n</code> is silently ignored — CH 11
6	B	Ownership means the promise survives; new name each time — CH 15/16
7	A	Pull failed: bad tag or private registry without credentials — CH 23
8	D	30000–32767, auto-assigned if omitted — CH 25
9	C	Watch & program iptables/IPVS — never forwards traffic itself — CH 27
10	A	NoExecute evicts now; NoSchedule/Prefer only block placement — CH 33

**9–10**

Exam-ready on this material

**6–8**Re-read the missed chapters'  boxes**<6**

Re-run the labs — then retake



Missed Q4 or Q10? Those are the two most common interview traps: quorum math and NoExecute eviction.
Missed Q5? The cluster-scoped list (Nodes, PVs, ClusterRoles, Namespaces) is a free exam point — memorize it.

[↑ back to top](#)



Cluster — Control Plane + Worker Nodes managed as one unit.

Control Plane — the brain: API Server, etcd, Scheduler, Controllers. Makes decisions; runs no user workloads (by taint).

API Server — the only door 🚪. All reads/writes (kubectl, kubelet, controllers) go through it; only it touches etcd.

etcd — the cluster's memory 💾. Needs quorum (majority) to write — why production runs 3 or 5 Control Planes.

Scheduler — assigns new Pods to Nodes based on resources, taints, affinity.

Controller — reconciliation loop: watch actual vs desired, act to close the gap.

kubelet — the node agent 🧑. Talks to the API Server, runs Pods via the runtime, reports status.

Container Runtime — pulls images and runs containers (containerd, CRI-O).

kube-proxy — watches Services/Endpoints, programs iptables/IPVS rules per node. Never forwards packets itself.

Pod — smallest deployable unit: 1+ containers sharing IP, volumes, lifecycle.

ReplicaSet — the guarantor: keeps N matching-labeled Pods alive via a template.

Deployment — manages ReplicaSets to enable versioned, zero-downtime rollouts and rollbacks.

Namespace — logical partition of names + RBAC + quotas inside one cluster. Apartments, not buildings.

Service — stable virtual IP + DNS name in front of a changing set of Pods.

NodePort — Service type opening one high port (30000–32767) on every node.

Taint / Toleration — node-side repulsion / pod-side permission. NoSchedule blocks placement; NoExecute also evicts.



Self-test the glossary: cover the right side, read each term, say the one-line definition aloud. If you can't — that's tonight's reading.

[↑ back to top](#)



⚙️ LIFECYCLE

```
$ kubectl create deploy web --image=nginx --replicas=3
$ kubectl get pods -w
$ kubectl scale deploy web --replicas=5
$ kubectl set image deploy/web nginx=nginx:1.26
$ kubectl rollout status|history|undo deploy/web
$ kubectl delete deploy web
```

🔍 DEBUG

```
$ kubectl describe pod P # events at BOTTOM
$ kubectl logs P --previous # crashed container
$ kubectl exec -it P -- sh
$ kubectl get events --sort-by=.lastTimestamp
Pending→scheduler · ImagePull→registry
CrashLoop→app
```

🌐 SERVICE TYPES

Type	Reach
ClusterIP	inside only (VIP + DNS)
NodePort	node:30000–32767 from outside
LoadBalancer	cloud LB → NodePorts

🚧 TAINTS

```
$ kubectl taint node N k=v:NoSchedule
$ kubectl taint node N k=v:NoExecute
$ kubectl taint node N k=v:NoSchedule- # remove
tolerations: key/op/value/effect
```

💡 FACTS WORTH A JOB OFFER

Cluster = CP + Workers · API Server is the only door · only it touches etcd

Quorum: 3 CP → tolerate 1 · **2 CP is the worst number** · odd counts only

Namespace partitions **names** , not hardware · Nodes are cluster-scoped

Standalone Pod dies = gone · owned Pod dies = replaced (new name)

Deployment → ReplicaSet(s) → Pods · rollback = new revision, not a rewind

ClusterIP is a fiction · kube-proxy programs rules, forwards nothing

One Pod = one IP · containers share localhost, volumes, and fate

[↑ back to top](#)



The whole book in one tree. Close this page, draw it on paper, then check yourself.

CLUSTER

- |— **CONTROL PLANE** ×3 (quorum)
 - |— **API Server**  ← kubectl · kubelet · controllers · scheduler
 - |— **etcd**  ← API Server only
 - |— **Scheduler**  Pod → Node placement
 - |— **Controllers**  desired ↔ actual, forever
- |— **NAMESPACES** frontend · ai · dev (+ 4 system ns)
 - |— **Deployments** → **ReplicaSets** → **Pods**  (+ Services · DNS)
- |— **WORKERS** 
 - |— **kubelet**  agent ↔ API Server
 - |— **Runtime**  containers
 - |— **kube-proxy**  VIP rules · DNAT · never forwards

✓ You can now...

Draw the two halves and name all 7 components with one verb each

Explain self-healing as a **reconciliation loop**, not magic

Do the quorum math and say why 2 Control Planes are the worst number

Trace `nginx:1.25 → 1.26 → broken → undo` through the ReplicaSet versions

Narrate one packet: DNS → VIP → DNAT → Pod → conntrack → home

Taint a node, watch a Pod go Pending, write the toleration that rescues it



Where to next: ConfigMaps · Secrets & RBAC (Book 02) · Services deep-dive & Ingress (Book 03) · Taints & scheduling in full (Book 04). The spine you just learned is the same spine in every one of them.

— End of Book 01 ·  —